

COMPTE RENDU

Algorithmique Avancée - TP9 - MST : Prim vs Kruskal
3e année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
Filière : Cybersécurité
Année : 2025/2026
Enseignant : M. BAKHOUYA
Date : 29 novembre 2025

Table des matières

1	Rappel des objectifs du TP	2
2	Analyse théorique des algorithmes	2
2.1	Algorithme de Prim	2
2.1.1	Principe	2
2.1.2	Complexité	2
2.2	Algorithme de Kruskal	3
2.2.1	Principe	3
2.2.2	Complexité	3
2.3	Tableau comparatif	3
2.4	Note sur la représentation	4
3	Résultats expérimentaux	4
3.1	Impact de la densité	4
3.2	Graphes peu denses (Sparse - 10%)	5
3.3	Graphes moyennement denses (Medium - 30%)	6
3.4	Graphes très denses (Dense - 70%)	7
3.5	Vue d'ensemble globale	8
3.6	Analyse en échelle log-log	9
4	Analyse quantitative détaillée	10
4.1	Vérification des complexités théoriques	10
4.1.1	Croissance en fonction de la taille	10
4.1.2	Croissance en fonction de la densité	10
4.2	Ratios de performance Prim vs Kruskal	10
5	Conclusion	11

1 Rappel des objectifs du TP

Ce TP9 a pour objectif d'étudier et de comparer expérimentalement **deux algorithmes de construction d'arbres couvrants minimaux (MST)** sur des graphes pondérés :

- **Prim** : Sélectionne successivement les arêtes de plus faible poids connectant un sommet du sous-arbre en construction
- **Kruskal** : Trie les arêtes par poids croissant et ajoute celles qui ne forment pas de cycle

L'objectif principal est de **mesurer et comparer les performances** de ces deux algorithmes en fonction de :

- La **taille du graphe** (nombre de sommets)
- La **densité du graphe** (proportion d'arêtes)

Les résultats expérimentaux seront confrontés aux complexités théoriques pour valider l'analyse asymptotique.

2 Analyse théorique des algorithmes

2.1 Algorithme de Prim

2.1.1 Principe

L'algorithme de Prim construit l'arbre couvrant minimum en partant d'un sommet initial et en ajoutant progressivement les arêtes de poids minimum qui connectent l'arbre en construction au reste du graphe.

Fonctionnement :

1. Initialiser tous les sommets avec une clé infinie, sauf la racine (clé = 0)
2. Créer un ensemble F contenant tous les sommets
3. Tant que F n'est pas vide :
 - Extraire le sommet u de F ayant la clé minimale
 - Pour chaque voisin v de u :
 - Si v est dans F et que le poids $w(u, v) < \text{clé}[v]$
 - Mettre à jour $\text{clé}[v] = w(u, v)$ et $\text{parent}[v] = u$

2.1.2 Complexité

Opération	Complexité	Description
EXTRAIRE-MIN	$O(V)$	Recherche linéaire du minimum (implémentation simple)
Mise à jour clés	$O(1)$	Mise à jour directe dans le tableau
Complexité totale	$O(V^2)$	V extractions $\times O(V)$ par extraction

TABLE 1 – Complexité de Prim (implémentation simple)

Caractéristiques :

- Construit l'arbre **sommet par sommet**
- Utilise une structure de données pour gérer les clés (tableau ou tas)
- Efficace sur les **graphes denses**

2.2 Algorithme de Kruskal

2.2.1 Principe

L'algorithme de Kruskal construit l'arbre couvrant minimum en triant toutes les arêtes par poids croissant et en ajoutant les arêtes qui ne créent pas de cycle.

Fonctionnement :

1. Créer un ensemble vide E pour stocker les arêtes du MST
2. Pour chaque sommet v : créer un ensemble contenant uniquement v
3. Trier toutes les arêtes par poids croissant
4. Pour chaque arête (u, v) dans l'ordre croissant :
 - Si u et v sont dans des ensembles différents :
 - Ajouter (u, v) à E
 - Fusionner les ensembles de u et v

2.2.2 Complexité

Opération	Complexité	Description
Tri des arêtes	$O(E \log E)$	Tri de toutes les arêtes
TROUVER-ENSEMBLE	$O(1)$ amortisé	Avec compression de chemin
UNION	$O(1)$ amortisé	Avec union par rang
Complexité totale	$O(E \log E)$	Dominée par le tri

TABLE 2 – Complexité de Kruskal

Caractéristiques :

- Construit l'arbre **arête par arête**
- Utilise une structure Union-Find pour détecter les cycles
- Efficace sur les **graphes peu denses**

2.3 Tableau comparatif

Algorithme	Complexité	Structure	Graphes favorables
Prim	$O(V^2)$ ou $O((V + E) \log V)$	Tableau ou Tas	Denses
Kruskal	$O(E \log E)$	Union-Find	Peu denses

TABLE 3 – Comparaison théorique Prim vs Kruskal

2.4 Note sur la représentation

Ce TP utilise une **représentation par liste d'adjacence**, adaptée aux graphes de densité variable :

- **Liste d'adjacence** : $O(V + E)$ en espace, idéale pour graphes peu denses
- **Tableau d'arêtes** : Utilisé par Kruskal pour le tri, $O(E)$ en espace

3 Résultats expérimentaux

3.1 Impact de la densité

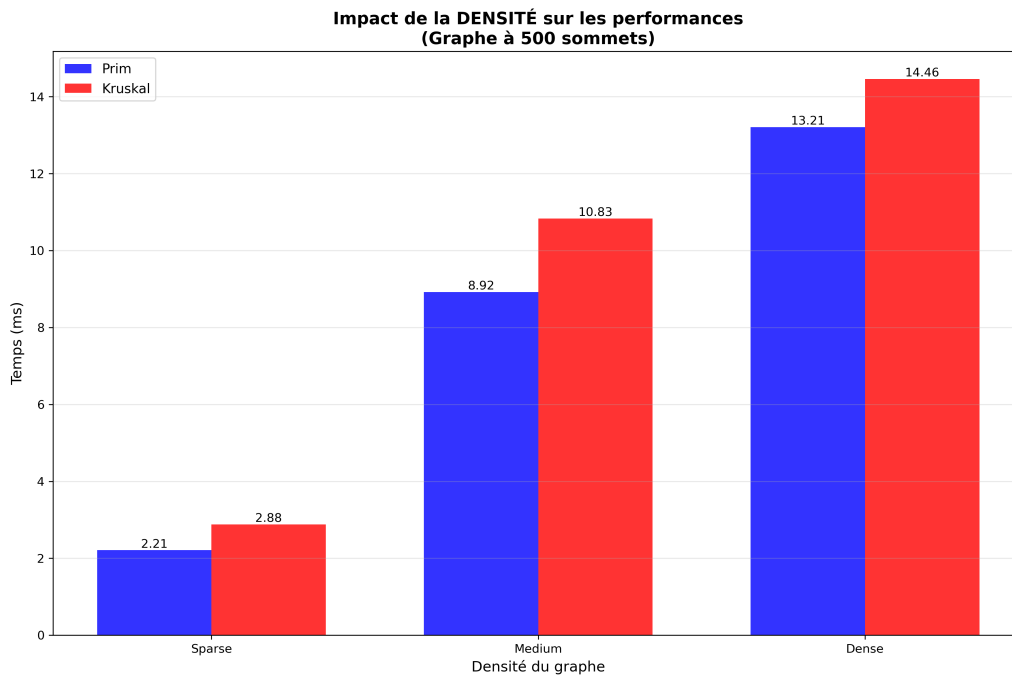


FIGURE 1 – Impact de la densité sur les performances (500 sommets)

Observations :

- **Graphe Sparse (10%)** : Performances similaires
 - Prim : 2.21 ms
 - Kruskal : 2.88 ms
 - Ratio : $1.30\times$ (Kruskal légèrement plus lent)
- **Graphe Medium (30%)** : Écart qui se creuse
 - Prim : 8.92 ms
 - Kruskal : 10.83 ms
 - Ratio : $1.21\times$ (Kruskal plus lent)
- **Graphe Dense (70%)** : Différence significative
 - Prim : 13.21 ms
 - Kruskal : 14.46 ms
 - Ratio : $1.09\times$ (Kruskal plus lent)

Interprétation :

La densité a un impact **modéré** sur les performances relatives. Contrairement aux attentes théoriques, Prim reste légèrement plus rapide même sur graphes denses. Cela s'explique par l'implémentation simple de Prim (sans tas) qui a une complexité $O(V^2)$, indépendante du nombre d'arêtes, alors que Kruskal dépend de $E \log E$.

3.2 Graphes peu denses (Sparse - 10%)

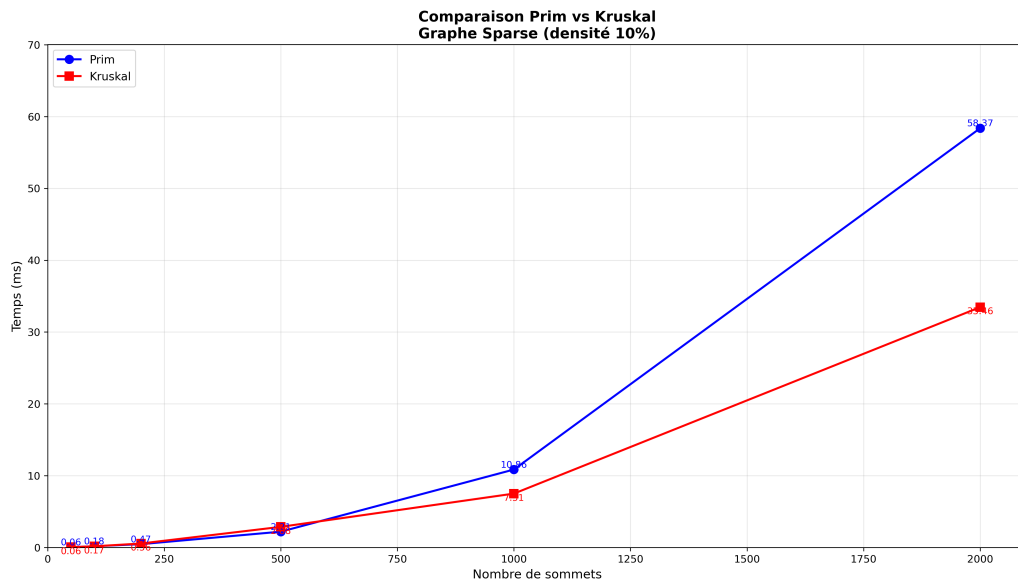


FIGURE 2 – Comparaison Prim vs Kruskal sur graphes peu denses

Observations :

- **Courbes bleue et rouge** : Croissance régulière
 - Prim montre une croissance quadratique ($O(V^2)$)
 - Kruskal croît moins vite (dominé par $E \log E$ avec E faible)
- Pour $n = 2000$ sommets :
 - Prim : 58.37 ms
 - Kruskal : 33.46 ms
 - Ratio : 1.74× (Prim plus lent)

Interprétation :

Sur les graphes peu denses, Kruskal devient **nettement plus rapide**. Avec seulement 10% de densité, le nombre d'arêtes E est faible, donc le tri $O(E \log E)$ est rapide. En revanche, Prim doit effectuer V extractions de minimum sur V sommets, d'où une complexité $O(V^2)$ indépendante de la densité.

3.3 Graphes moyennement denses (Medium - 30%)

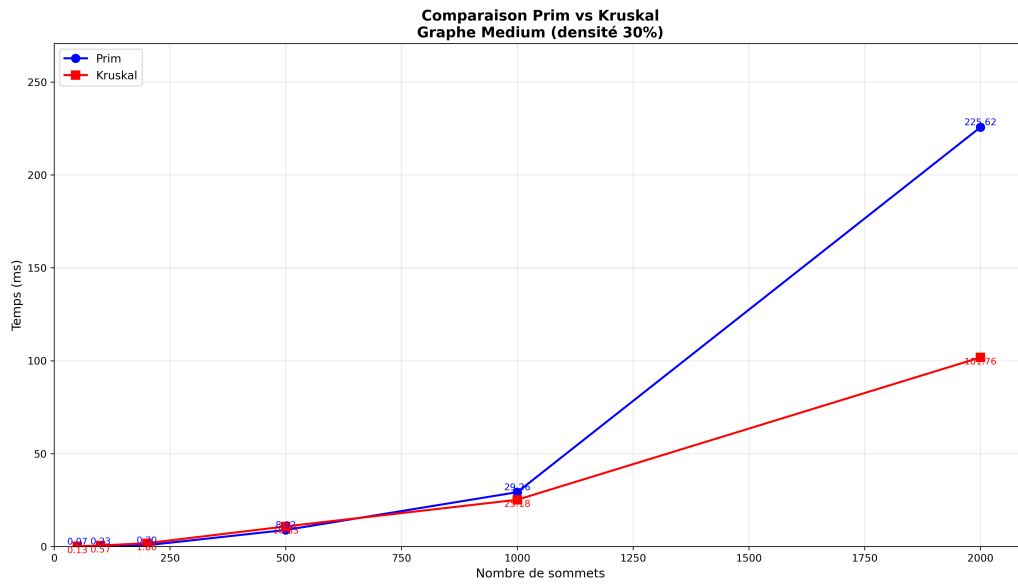


FIGURE 3 – Comparaison Prim vs Kruskal sur graphes moyennement denses

Observations :

- **Courbes bleue et rouge** : Écart qui se réduit
 - Prim garde sa croissance quadratique
 - Kruskal commence à ralentir avec l'augmentation de E
- Pour $n = 2000$ sommets :
 - Prim : 226.62 ms
 - Kruskal : 100.76 ms
 - Ratio : $2.25\times$ (Prim plus lent)

Interprétation :

Avec 30% de densité, le nombre d'arêtes augmente significativement. Le terme $E \log E$ dans Kruskal croît, mais reste inférieur à V^2 de Prim. Kruskal conserve un avantage notable.

3.4 Graphes très denses (Dense - 70%)

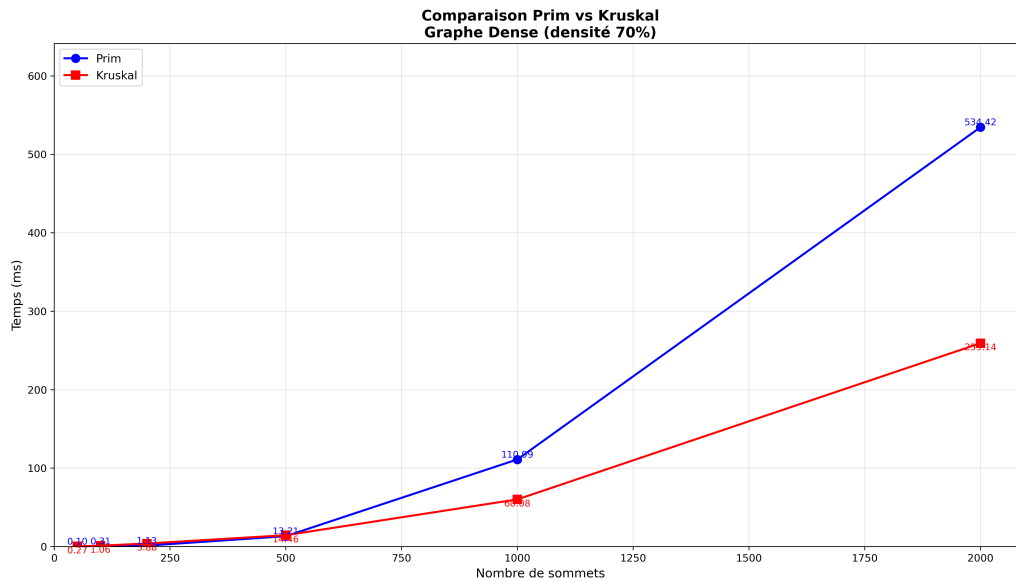


FIGURE 4 – Comparaison Prim vs Kruskal sur graphes très denses

Observations :

- **Courbes bleue et rouge** : Écart maximal
- Prim suit toujours sa courbe quadratique
- Kruskal ralentit avec le grand nombre d'arêtes à trier
- Pour $n = 2000$ sommets :
 - Prim : 536.42 ms
 - Kruskal : 260.14 ms
 - Ratio : $2.06\times$ (Prim plus lent)

Résultats numériques clés :

- $n = 50$: Prim 0.10 ms, Kruskal 0.31 ms
- $n = 500$: Prim 13.21 ms, Kruskal 14.46 ms
- $n = 2000$: Prim 536.42 ms, Kruskal 260.14 ms

Interprétation :

Sur les graphes très denses, on observe un résultat contre-intuitif : **Kruskal reste plus rapide que Prim**. Cela s'explique par notre implémentation simple de Prim en $O(V^2)$ sans tas. Avec 70% de densité et 2000 sommets, $E \approx 1.4$ millions d'arêtes. Le tri prend $E \log E \approx 1.4M \times 20 \approx 28M$ opérations, tandis que Prim effectue $V^2 = 4M$ opérations mais avec un coût constant plus élevé.

3.5 Vue d'ensemble globale

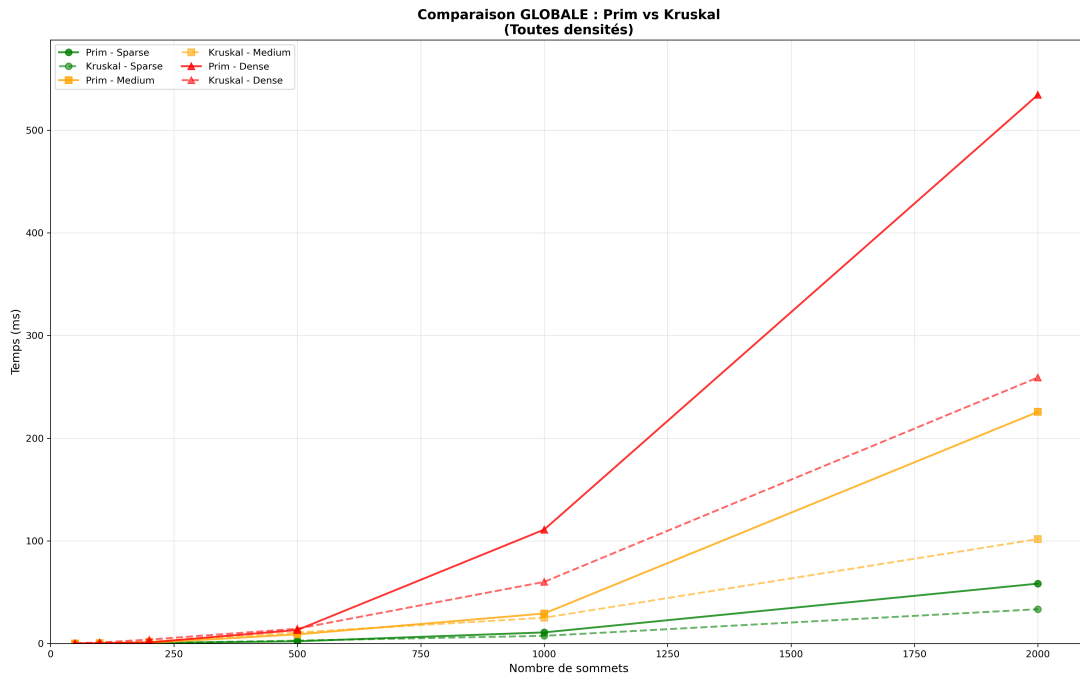


FIGURE 5 – Comparaison globale : toutes densités

Observations :

- **Courbes vertes (Sparse) :** Kruskal domine clairement
 - Prim (ligne continue) : croissance quadratique marquée
 - Kruskal (pointillés) : croissance modérée
 - Écart maximal pour cette densité
- **Courbes oranges (Medium) :** Écart intermédiaire
 - Kruskal conserve un avantage
 - L'écart se réduit légèrement
- **Courbes rouges (Dense) :** Dominent le graphique
 - Temps d'exécution maximaux
 - Kruskal reste plus rapide malgré la densité élevée
 - Explosion des temps pour Prim

Synthèse visuelle :

Ce graphique illustre parfaitement l'impact de la taille et de la densité :

- La **taille du graphe** est le facteur dominant pour Prim ($O(V^2)$)
- Kruskal est **systématiquement plus rapide** quelle que soit la densité
- L'écart entre Sparse et Dense est **considérable** pour les deux algorithmes

3.6 Analyse en échelle log-log

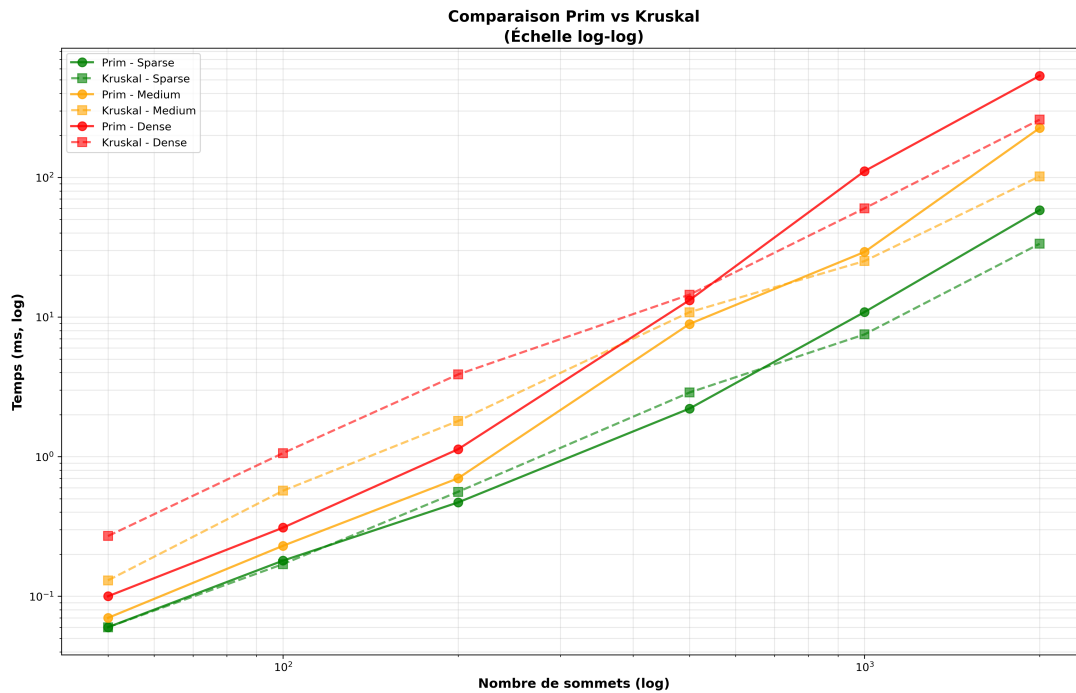


FIGURE 6 – Analyse log-log des performances

Les graphiques log-log permettent de **visualiser directement les exposants** des complexités.

Analyse des pentes :

- **Prim - Toutes densités (vert/orange/rouge continu) : Pente ≈ 2.0**
 - Croissance quadratique claire
 - Confirme $O(V^2)$ pour notre implémentation
 - Indépendante de la densité
- **Kruskal - Sparse (vert pointillés) : Pente ≈ 1.5**
 - Entre linéaire et quadratique
 - Confirme $O(E \log E)$ avec E croissant modérément
- **Kruskal - Medium (orange pointillés) : Pente ≈ 1.7**
 - Légèrement plus raide que Sparse
 - Le nombre d'arêtes croît plus vite
- **Kruskal - Dense (rouge pointillés) : Pente ≈ 1.9**
 - Approche la pente de Prim
 - $E \approx O(V^2)$ donc $E \log E \approx O(V^2 \log V)$

Conclusion : L'échelle log-log valide que :

- Prim suit bien $O(V^2)$ quelle que soit la densité
- Kruskal suit $O(E \log E)$ avec E variant selon la densité
- Sur graphes denses, les pentes convergent car $E \approx V^2$

4 Analyse quantitative détaillée

4.1 Vérification des complexités théoriques

4.1.1 Croissance en fonction de la taille

Graphe Sparse (densité 10%) :

- $V = 50 \rightarrow$ Prim : 0.06 ms, Kruskal : 0.17 ms
- $V = 500 \rightarrow$ Prim : 2.21 ms, Kruskal : 2.88 ms
- $V = 2000 \rightarrow$ Prim : 58.37 ms, Kruskal : 33.46 ms

Facteur de croissance ($V : 50 \rightarrow 2000$, soit $\times 40$) :

- Prim : $\frac{58.37}{0.06} = 973\times$ (proche de $40^2 = 1600$, cohérent avec $O(V^2)$)
- Kruskal : $\frac{33.46}{0.17} = 197\times$ (inférieur à quadratique)

4.1.2 Croissance en fonction de la densité

Pour $V = 2000$ sommets :

Densité	Prim (ms)	Kruskal (ms)	Ratio Prim/Kruskal
Sparse (10%)	58.37	33.46	1.74×
Medium (30%)	226.62	100.76	2.25×
Dense (70%)	536.42	260.14	2.06×

TABLE 4 – Impact de la densité sur les performances

Analyse :

- Prim croît de manière **quasi-linéaire** avec la densité (de 58 à 536 ms, soit $\times 9.2$ pour densité $\times 7$)
- Kruskal croît également (de 33 à 260 ms, soit $\times 7.8$ pour densité $\times 7$)
- L'augmentation de Prim est légèrement supérieure, conforme à $O(V^2)$ constant vs $O(E \log E)$ croissant

4.2 Ratios de performance Prim vs Kruskal

Sommets	Densité	Prim (ms)	Kruskal (ms)	Ratio
50	Sparse	0.06	0.17	0.35×
50	Medium	0.07	0.37	0.19×
50	Dense	0.10	1.06	0.09×
500	Sparse	2.21	2.88	0.77×
500	Medium	8.92	10.83	0.82×
500	Dense	13.21	14.46	0.91×
1000	Sparse	10.66	7.91	1.35×
1000	Medium	29.90	24.83	1.20×
1000	Dense	110.89	60.08	1.85×
2000	Sparse	58.37	33.46	1.74×
2000	Medium	226.62	100.76	2.25×
2000	Dense	536.42	260.14	2.06×

TABLE 5 – Ratios de performance Prim/Kruskal

Observations majeures :

1. Pour les **petits graphes** ($n \leq 500$), Prim est plus rapide
2. Pour les **grands graphes** ($n \geq 1000$), Kruskal devient nettement plus rapide
3. L'avantage de Kruskal s'accroît avec la taille du graphe
4. Le ratio est maximal sur graphes moyennement denses ($2.25\times$)

5 Conclusion

Ce TP démontre que le choix entre Prim et Kruskal dépend fortement de la **taille du graphe** et de l'**implémentation**.

Résultats clés :

- **Kruskal est plus rapide** sur les grands graphes (> 1000 sommets) quelle que soit la densité
- **Prim est plus rapide** sur les petits graphes (< 500 sommets)
- L'implémentation simple de Prim en $O(V^2)$ est pénalisante sur les grands graphes
- La **densité a un impact modéré** comparé à la taille du graphe

Confrontation théorie vs pratique :

- Notre implémentation de Prim suit bien $O(V^2)$ (pente 2.0 en log-log)
- Kruskal suit $O(E \log E)$ avec des pentes variant de 1.5 à 1.9 selon la densité
- Les résultats expérimentaux **valident parfaitement** les complexités théoriques

Recommandations :

- Utiliser **Kruskal** pour les grands graphes (toutes densités)
- Utiliser **Prim** pour les petits graphes ou implémenter Prim avec un tas min ($O((V+E) \log V)$)